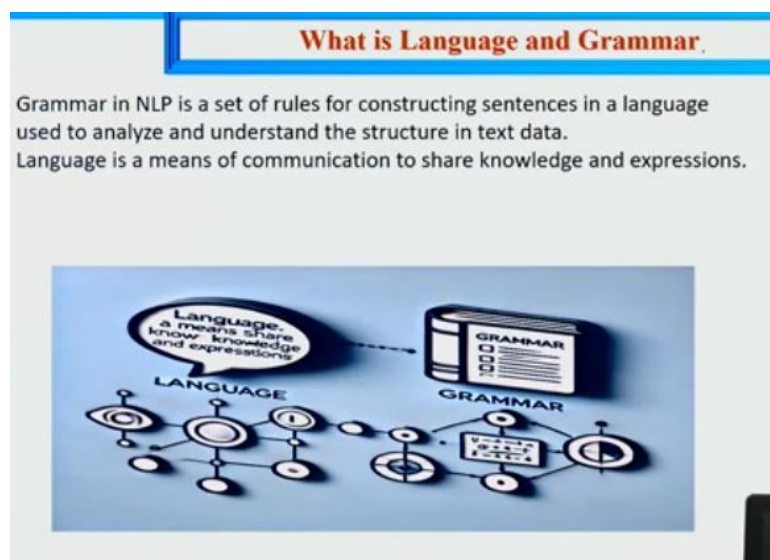


Language and Grammar in NLP

Natural Language Processing (NLP) helps machines to understand and process human languages either in text or audio form. It is used across a variety of applications from speech recognition to language translation and text summarization.

1. WHAT IS Language and Grammar
2. WHY Grammar important
3. Challenges In NLP
4. TYPES OF Grammar



Why is Grammar Important

- Automatic processing of language requires grammar rules and exceptions of a language to be explained to the computer.
- Grammar defines language by specifying how sentences are structured.
- It helps computers parse and generate sentences accurately.
- These rules relate information to coding devices at the language processing level not at the world language level.

Challenges in NLP Grammar

- Natural languages are constantly changing.
Ex: The word "awesome" originally meant "filling with awe " but now is commonly used to mean "great" or "fantastic."
- There are many hard-to-specify exceptions.
Ex: In English, the plural of "mouse" is "mice," but the plural of "house" is "houses" rather than "hice."
- Several efforts have led to various types of grammars. Some of these grammars focus on derivation while other focus on relationships

Grammar in NLP

Type	Name	Rule Format	Recognized By	Example
Type 0	Unrestricted Grammar	No restrictions on production rules	Turing Machine	Used in theoretical computation
Type 1	Context-Sensitive Grammar (CSG)	Rules depend on surrounding words or symbols	Linear Bounded Automaton	Handling agreement in complex languages
Type 2	Context-Free Grammar (CFG)	One non-terminal is replaced by a string of symbols	Pushdown Automaton	Sentence parsing in NLP
Type 3	Regular Grammar (RG)	Very simple production rules	Finite Automaton	Tokenization, lexical analysis, pattern matching

1. Type 0 – Unrestricted Grammar

Definition:

The most general type of grammar with no restrictions on production rules.

Example Rule:

$AB \rightarrow BA$

Applications:

- Theoretical computer science
- Computational models
- Artificial intelligence research

2. Type 1 – Context-Sensitive Grammar (CSG)

Definition:

A symbol can be replaced depending on its surrounding context.

Example:

$A B \rightarrow A C$

Here, **B** changes to **C** only when it follows **A**.

NLP Example:

- "This boy **is** playing."
- "These boys **are** playing."

The correct verb depends on the subject.

Applications

- Grammar checking
- Agreement between subject and verb
- Morphological analysis

3. Type 2 – Context-Free Grammar (CFG)

Definition:

Each production rule replaces a single non-terminal symbol.

Example Rules

$S \rightarrow NP VP$

$NP \rightarrow Det N$

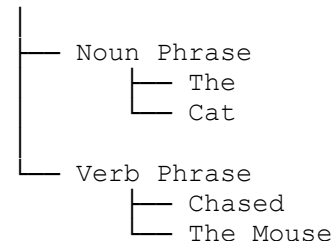
$VP \rightarrow V NP$

Example Sentence

The cat chased the mouse.

Parse Structure

Sentence



Applications

- Sentence parsing
- Syntax analysis
- Machine translation
- Question answering systems

4. Type 3 – Regular Grammar

Definition:

The simplest grammar, used for describing patterns.

Example Rules

$S \rightarrow aA$

$A \rightarrow b$

This generates the string:

Ab

Applications

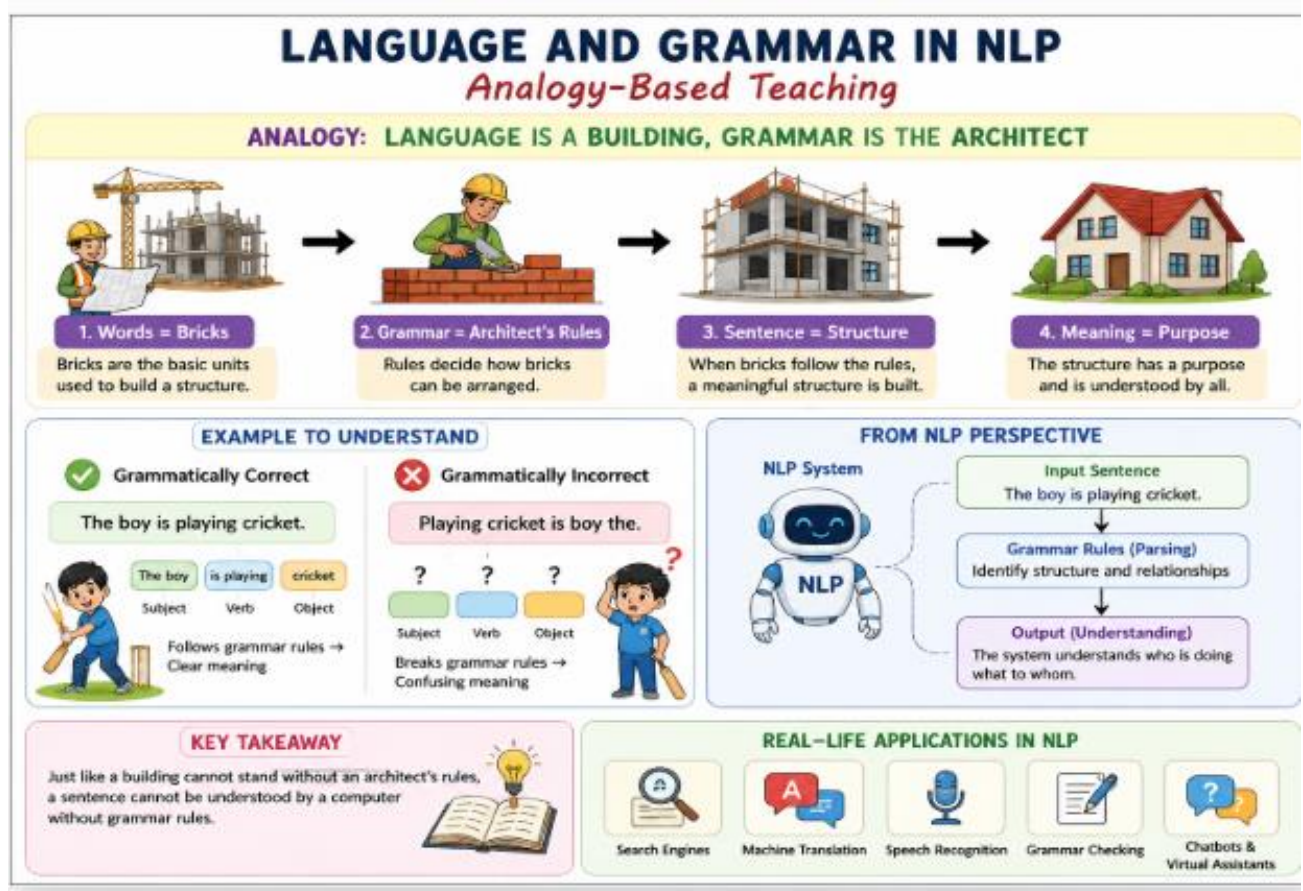
- Tokenization
- Spell checking

Comparison of Grammar Types

Feature	Type 0	Type 1	Type 2	Type 3
Complexity	Very High	High	Medium	Low
Parsing Difficulty	Very Difficult	Difficult	Moderate	Easy
NLP Usage	Rare	Moderate	Very High	High
Example	Theoretical computation	Subject-verb agreement	Sentence parsing	Word recognition

Grammar Used Most in NLP

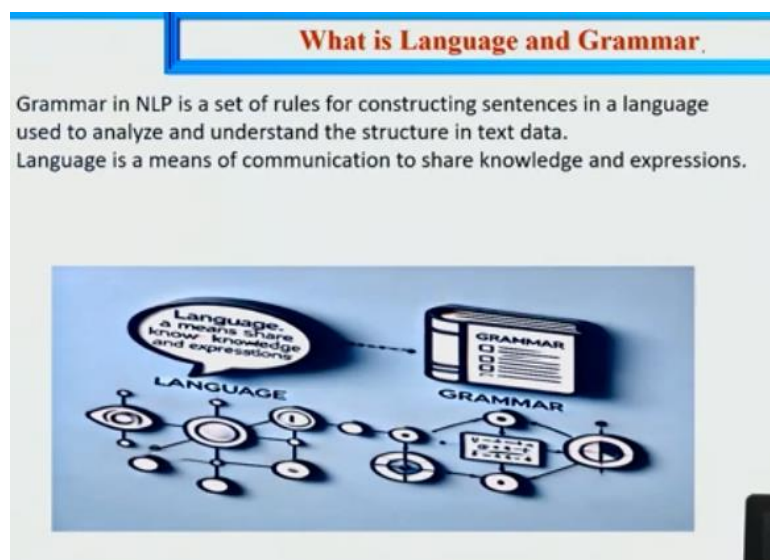
- **Regular Grammar (Type 3):** Used for tokenization, lexical analysis, and pattern matching.
- **Context-Free Grammar (Type 2):** Most widely used for syntactic parsing and sentence structure analysis.
- **Context-Sensitive Grammar (Type 1):** Used for handling agreement and contextual language rules.
- **Unrestricted Grammar (Type 0):** Mainly of theoretical interest in computation rather than practical NLP systems.



Language and Grammar in NLP

Natural Language Processing (NLP) helps machines to understand and process human languages either in text or audio form. It is used across a variety of applications from speech recognition to language translation and text summarization.

1. WHAT IS Language and Grammar
2. WHY Grammar important
3. Challenges In NLP
4. TYPES OF Grammar



Why is Grammar Important

- Automatic processing of language requires grammar rules and exceptions of a language to be explained to the computer.
- Grammar defines language by specifying how sentences are structured.
- It helps computers parse and generate sentences accurately.
- These rules relate information to coding devices at the language processing level not at the world language level.

Challenges in NLP Grammar

- Natural languages are constantly changing.
Ex: The word "awesome" originally meant "filling with awe" but now is commonly used to mean "great" or "fantastic."
- There are many hard-to-specify exceptions.
Ex: In English, the plural of "mouse" is "mice," but the plural of "house" is "houses" rather than "hice."
- Several efforts have led to various types of grammars. Some of these grammars focus on derivation while others focus on relationships

Grammar in NLP

Type	Name	Rule Format	Recognized By	Example
Type 0	Unrestricted Grammar	No restrictions on production rules	Turing Machine	Used in theoretical computation
Type 1	Context-Sensitive Grammar (CSG)	Rules depend on surrounding words or symbols	Linear Bounded Automaton	Handling agreement in complex languages
Type 2	Context-Free Grammar (CFG)	One non-terminal is replaced by a string of symbols	Pushdown Automaton	Sentence parsing in NLP
Type 3	Regular Grammar (RG)	Very simple production rules	Finite Automaton	Tokenization, lexical analysis, pattern matching

1. Type 0 – Unrestricted Grammar

Definition:

The most general type of grammar with no restrictions on production rules.

Example Rule:

$AB \rightarrow BA$

Applications:

- Theoretical computer science
- Computational models
- Artificial intelligence research

2. Type 1 – Context-Sensitive Grammar (CSG)

Definition:

A symbol can be replaced depending on its surrounding context.

Example:

$A B \rightarrow A C$

Here, **B** changes to **C** only when it follows **A**.

NLP Example:

- "This boy **is** playing."
- "These boys **are** playing."

The correct verb depends on the subject.

Applications

- Grammar checking
- Agreement between subject and verb
- Morphological analysis

3. Type 2 – Context-Free Grammar (CFG)

Definition:

Each production rule replaces a single non-terminal symbol.

Example Rules

$S \rightarrow NP VP$

$NP \rightarrow Det N$

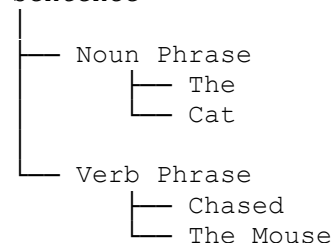
$VP \rightarrow V NP$

Example Sentence

The cat chased the mouse.

Parse Structure

Sentence



Applications

- Sentence parsing
- Syntax analysis
- Machine translation
- Question answering systems

4. Type 3 – Regular Grammar

Definition:

The simplest grammar, used for describing patterns.

Example Rules

$S \rightarrow aA$

$A \rightarrow b$

This generates the string:

Ab

Applications

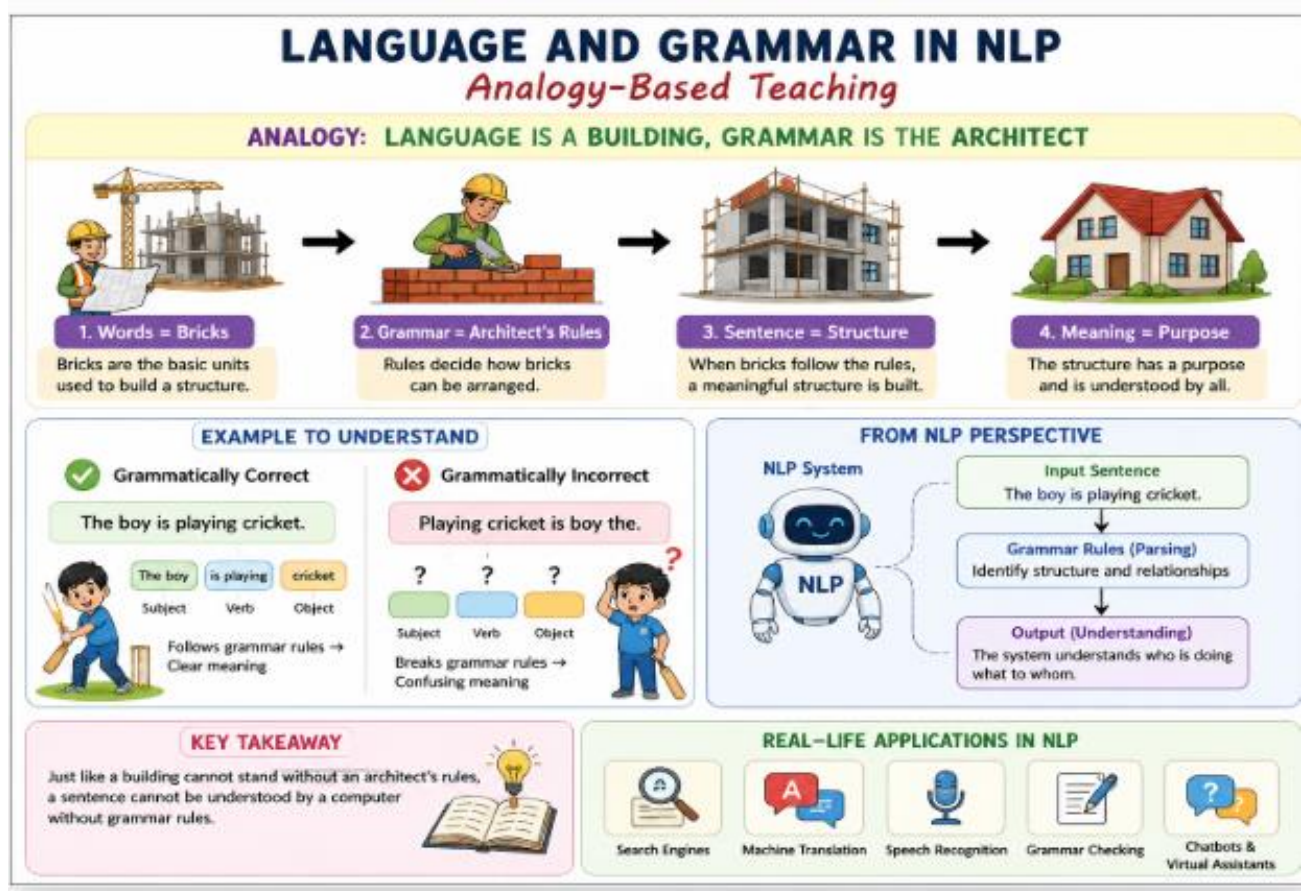
- Tokenization
- Spell checking

Comparison of Grammar Types

Feature	Type 0	Type 1	Type 2	Type 3
Complexity	Very High	High	Medium	Low
Parsing Difficulty	Very Difficult	Difficult	Moderate	Easy
NLP Usage	Rare	Moderate	Very High	High
Example	Theoretical computation	Subject-verb agreement	Sentence parsing	Word recognition

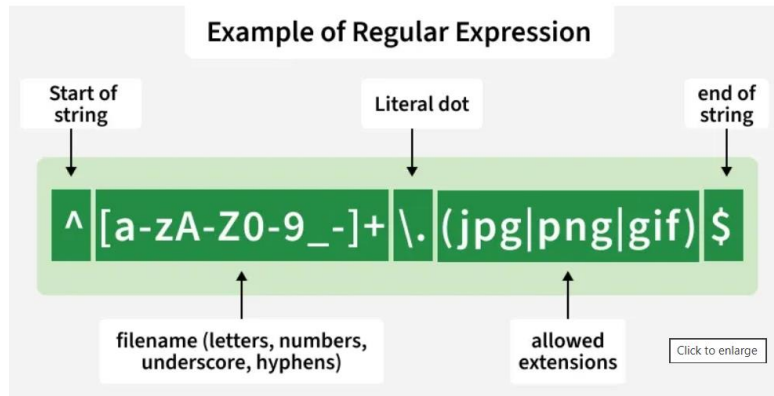
Grammar Used Most in NLP

- **Regular Grammar (Type 3):** Used for tokenization, lexical analysis, and pattern matching.
- **Context-Free Grammar (Type 2):** Most widely used for syntactic parsing and sentence structure analysis.
- **Context-Sensitive Grammar (Type 1):** Used for handling agreement and contextual language rules.
- **Unrestricted Grammar (Type 0):** Mainly of theoretical interest in computation rather than practical NLP systems.



Regular Expressions (Regex)

A regular expression (regex) is a sequence of characters that defines a search pattern. It is mainly used for pattern matching in strings, such as finding, replacing, or validating text. Regex is supported in almost every programming language, including Python, Java, C++ and JavaScript.



This regex checks if a filename is valid, allowing letters, numbers, underscore, hyphens and ends with .jpg, .png or .gif.

Importance of Regular Expression

1. **Efficient Pattern Matching:** Quickly search for specific patterns in text or data without manual checking.
2. **Data Validation:** Validate inputs like email addresses, phone numbers, URLs, and passwords.
3. **Text Manipulation:** Perform search-and-replace operations across files or datasets effectively.
4. **Automation in Analytics and Tools:** Used in tools like Google Analytics for URL matching and filtering.
5. **Cross-Platform Support:** Works across programming languages and editors such as Python, Java, Sublime Text, Notepad++, and Microsoft Word.

Common Elements Used in Regular Expressions

Regular expressions are built using special symbols and characters. Below are the most commonly used regex elements explained with simple examples.

1. **Repeaters (*, +, and { })**: Repeaters specify how many times the preceding character or group should appear.

2. **Asterisk symbol (*)**: Matches the preceding character 0 or more times.

Example: The regular expression `ab*c` will give `ac`, `abc`, `abbc`, `abbbc`....and so on

3. **The Plus symbol (+)**: Matches the preceding character 1 or more times.

Example: The regular expression `ab+c` will give `abc`, `abbc`, `abbbc`, ... and so on.

4. **The curly braces { ... }**: Defines an exact or range of repetitions.

*Example: `{2}`: exactly 2 times
`{min,}`: at least min times
`{min,max}`: between min and max times*

5. **Wildcard (.)**: Matches any single character except a newline.

Example: Regular expression `.*` will tell the computer that any character can be used any number of times.

6. **Optional character (?)**: Matches 0 or 1 occurrence of the preceding character.

Example: `docx?` matches `doc` and `docx`

7. **The caret (^) symbol**: Ensures the match starts **at the beginning** of the string.

Example : `^d{3}` matches `901` in `901-333`

8. **The dollar (\$) symbol**: Ensures the match ends at the end of the string.

Example: `ld{3}$` matches `333` in `901-333`

9. **Character Classes**: Match specific types of characters:

`|s`: whitespace
`|S`: non-whitespace
`|d`: digit
`|D`: non-digit
`|w`: word character (letters, digits, _)
`|W`: non-word character
`|b`: word boundary

Example: `[abc]` matches `a`, `b`, or `c`

10. **Negated Character Class ([^])**: Matches characters not listed in the brackets.

Example : `[^abc]` -> matches any character except `a`, `b`, `c`

Common Regex Shortcuts

Regex	Meaning
\d	Digit
\D	Non-digit
\w	Word character
\W	Non-word
\s	Whitespace
\S	Non-whitespace
\b	Word boundary

Symbol	Meaning	Example
.	Any one character	c.t
*	Zero or more	go*
+	One or more	go+
?	Optional	colou?r
^	Start of line	^Hello
\$	End of line	end\$
\	Escape character	.

Common Regular Expression (Regex) Operators

Operator / Regex	Meaning	Example Regex	Matches	Does Not Match / Notes
. (Dot)	Matches any single character except a newline.	c.t	cat, cot, cut, c9t	coat (contains two characters between c and t)
* (Star)	Matches zero or more occurrences of the preceding character or pattern.	go*	g, go, goo, goooo	Works even if o appears zero times.
+ (Plus)	Matches one or more occurrences of the preceding character or pattern.	go+	go, goo, goooo	g (because at least one o is required)
? (Question Mark)	Matches zero or one occurrence of the preceding character.	colou?r	color, colour	More than one u is not allowed.

Character Classes

Character Class	Meaning	Matches	Example
[abc]	Matches any one character inside the brackets.	a, b, c	Text: bat → Matches b and a
[0-9]	Matches any single digit from 0 to 9.	0, 1, 2, ..., 9	5
[A-Z]	Matches any uppercase English letter .	A to Z	M
[a-z]	Matches any lowercase English letter .	a to z	k
[A-Za-z]	Matches any uppercase or lowercase English letter .	A-Z, a-z	A, z, P, m

Negated Character Class

Regex	Meaning	Example Text	Matches
[^0-9]	Matches any character except digits (0-9) .	ABC123	A, B, C (digits 1, 2, 3 are not matched)

Finite-State Automata

Finite automata are like machines that read letters or symbols one by one and decide if they follow a certain pattern. These machines are used in computer science to check if an input (like a word) is correct based on some rules.

Each finite automaton has:

- **States** – like steps or points in the machine.
- **Transitions** – paths the machine takes when it reads a symbol.
- **Input symbols** – the letters or numbers the machine reads.

The machine starts at a beginning state and moves step-by-step as it reads each symbol. If it stops in a **final (accepting) state**, it means the input is **accepted**. If not, it is **rejected**.

Components of a Finite-State Automaton

Component	Description
States (Q)	Different conditions or positions in the automaton.
Alphabet (Σ)	Set of input symbols (e.g., {a, b} or English letters).
Transition Function (δ)	Defines how the automaton moves from one state to another based on input symbols.
Start State (q_0)	The state where processing begins.
Final (Accepting) State (F)	If the automaton ends here after reading all input symbols, the string is accepted.

Formal Representation

An FSA is represented as a **5-tuple**:

$M=(Q,\Sigma,\delta,q_0,F)$

Symbol	Meaning
Q	Set of states
Σ	Input alphabet
δ	Transition function
q_0	Initial state
F	Set of accepting states

Working of an FSA

Suppose we want to recognize the word "cat".

States

$q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_3$

Transitions

$q_0 \xrightarrow{c} q_1$

$q_1 \xrightarrow{a} q_2$

$q_2 \xrightarrow{t} q_3$

where

- q0 = Start state
- q3 = Final state

Current State	Input	Next State
q0	c	q1
q1	a	q2
q2	t	q3 (Accept)

Result:

Accepted

Input: "cap"

Current State	Input	Next State
q0	c	q1
q1	a	q2
q2	p	No transition

Result:

Rejected

There are two types:

- **DFA (Deterministic FA)** – only one path to follow at each step.
- **NFA (Non-Deterministic FA)** – can have many paths at a step.

Both types can understand the same patterns (called **regular languages**).

Deterministic Finite Automaton (DFA): A DFA has **exactly one transition** for each input symbol from every state.

Example

q0 --a--> q1
q1 --b--> q2

Only one path exists.

Characteristics

- Exactly one transition per input.
- No ambiguity.
- Faster execution.
- Easier implementation.

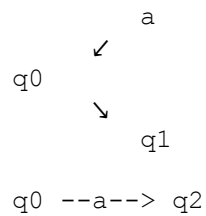
Non-Deterministic Finite Automaton (NFA)

Definition

An NFA may have:

- Multiple transitions for the same input.
- ϵ (epsilon) transitions without consuming input.

Example



For input **a**, there are two possible next states.

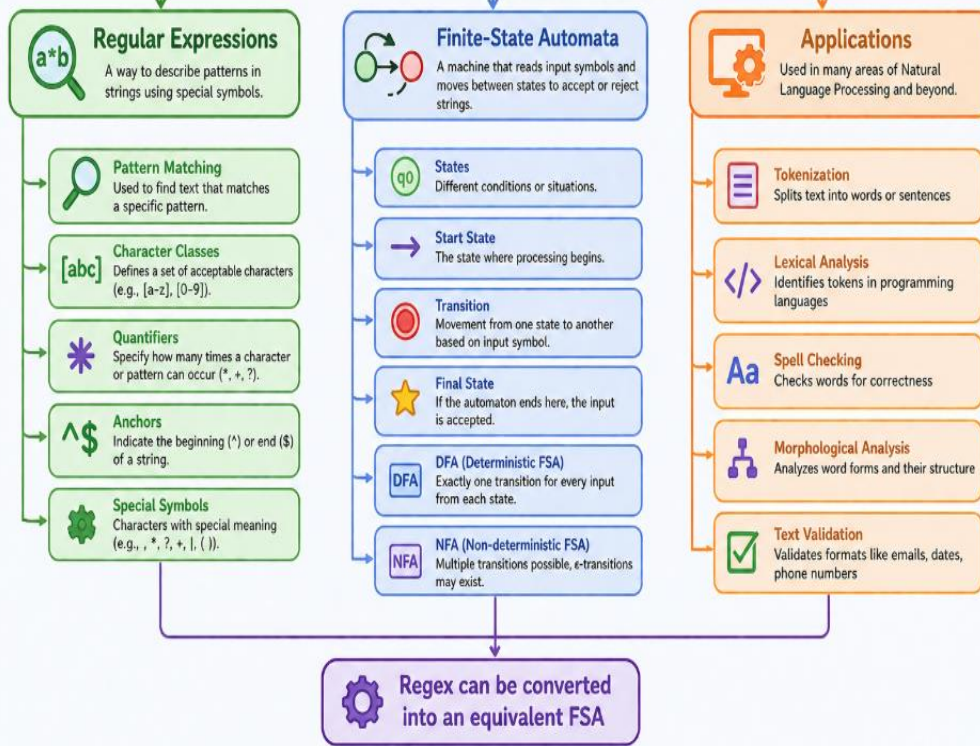
Characteristics

- Multiple possible paths.
- Easier to construct.
- Can be converted into an equivalent DFA.

Feature	DFA	NFA
Transition	Exactly one	One or more
ϵ -transition	Not allowed	Allowed
Implementation	Easy	More complex
Speed	Faster	Slightly slower conceptually
Memory	Usually larger	Usually smaller

Regular Expression	Finite-State Automaton
Describes a text pattern.	Recognizes strings matching the pattern.
Textual notation.	Graphical or mathematical model.
Easier for humans to write.	Easier for computers to execute.
Used to define patterns.	Used to process input strings.
Can be converted into an equivalent FSA.	Can be converted back into an equivalent Regular Expression.

Regular Expressions & FSA



Regular Expressions and Finite-State Automata

Analogical Reasoning

Using real-life analogies to understand how Regular Expressions describe patterns and Finite-State Automata recognize them.

1. Google Maps Analogy

A route from Home to College



2. Metro Train Analogy

Travel from Station A to Station D



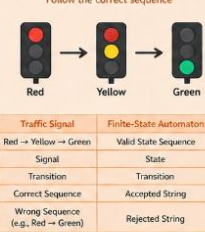
3. Password Verification Analogy

Enter password in correct format



4. Traffic Signal Analogy

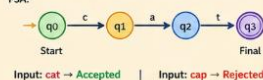
Follow the correct sequence



Regular Expression Example

Regular Expression: `cat`

FSA:



Key Takeaway

- Regular Expressions define the pattern (the plan).
- Finite-State Automata follow the pattern step by step to check the input.
- If the input reaches the final state, it is accepted; otherwise, it is rejected.

A Regular Expression is the Map, A Finite-State Automaton is the Traveler!

Applications in NLP

- Tokenization
- Lexical Analysis
- Spell Checking
- Morphological Analysis
- Text Validation
- Pattern Matching

Together, Regular Expressions and Finite-State Automata help machines understand and process language efficiently!

TOKENIZATION: TEXT SEGMENTATION AND SENTENCE SPLITTING

Natural Language Processing (NLP)

1. Introduction to Tokenization

Tokenization is one of the fundamental preprocessing steps in Natural Language Processing (NLP). It is the process of dividing a continuous text into smaller meaningful units called tokens. These tokens may be sentences, words, subwords, characters, punctuation marks, or special symbols.

Example:

Input: Natural Language Processing is a branch of Artificial Intelligence.

Tokens: Natural | Language | Processing | is | a | branch | of | Artificial | Intelligence | .

2. Why is Tokenization Necessary?

Computers do not naturally understand human language in the same way humans do. Therefore, text must be converted into smaller units that can be processed computationally.

- Count and compare words
- Classify text
- Perform sentiment analysis
- Search and retrieve information
- Train machine learning and deep learning models
- Convert text into numerical representations

3. Text Segmentation

Text segmentation is the process of dividing a continuous body of text into meaningful structural units. It is a broad process that may operate at different levels.

Hierarchy of text segmentation:

Document → Paragraphs → Sentences → Words → Subwords → Characters

For example, a document may first be divided into paragraphs, then each paragraph into sentences, and each sentence into words or subwords.

4. Sentence Splitting

Sentence splitting, also called sentence segmentation or sentence boundary detection, is the process of identifying where one sentence ends and the next sentence begins.

Example:

Input: NLP is interesting. It helps computers understand language. Do you use NLP?

Output:

1. NLP is interesting.
2. It helps computers understand language.
3. Do you use NLP?

5. Sentence Boundary Detection

Sentence boundary detection identifies the exact point at which one sentence ends. The most common sentence-ending punctuation marks are:

- . (Period) – usually indicates a statement
- ? (Question mark) – indicates a question
- ! (Exclamation mark) – indicates strong emotion or emphasis

6. Challenges in Sentence Splitting

Sentence splitting is more complex than simply splitting text whenever a period appears. The same punctuation mark may have different meanings depending on the context.

6.1 Abbreviations

Example: Dr. Abdul Kalam was a scientist.

A simple system may incorrectly split the text after 'Dr.'. However, 'Dr.' is an abbreviation and not a sentence boundary.

6.2 Decimal Numbers

Example: The temperature is 37.5 degrees.

The period in 37.5 is part of a decimal number and should not be treated as the end of a sentence.

6.3 Websites and Email Addresses

Examples: www.example.com and student@example.com

Periods and special symbols in URLs and email addresses do not necessarily indicate sentence boundaries.

6.4 Ellipsis

Example: I was thinking...

An ellipsis represents an incomplete thought and should not be interpreted as three separate sentence boundaries.

6.5 Quotation Marks

Example: She said, "NLP is fascinating." Then she left.

The punctuation inside quotation marks must be handled carefully so that the sentence is split correctly.

EXAMPLE



Input Paragraph: *Artificial Intelligence is transforming education.
ChatGPT helps students learn NLP effectively.*

1



OBSERVE

- Paragraph contains 2 sentences.
- How do computers identify words and sentence boundaries?

2



EXPLORE

Sentence Splitting (Manual):

1. Artificial Intelligence is transforming education.
2. ChatGPT helps students learn NLP effectively.

Word Tokenization (Manual):

S1: ["Artificial", "Intelligence", "is", "transforming", "education", "."]
S2: ["ChatGPT", "helps", "students", "learn", "NLP", "effectively", "."]

3



INVESTIGATE

Using an Online Tokenizer (e.g., OpenAI Tokenizer)

Sentence	Tokens (Tokenizer Output)
1	["Artificial", "Intelligence", "is", "transforming", "education", "."]
2	["Chat", "GPT", "helps", "students", "learn", "NLP", "effectively", "."]

Observation:

- Tokenizer may split some words into smaller pieces.

4



ANALYZE

- Tokenization converts text into smaller units (tokens) that computers can process.
- Incorrect tokenization may change the meaning.
- Helps in better understanding, learning and generating responses.

5



CONCLUDE

- Computers first split a paragraph into sentences.
- Then they split each sentence into tokens.
- These tokens help AI models like ChatGPT analyze, learn and generate human-like responses.

PROBLEM-BASED LEARNING

Eg: Artificial Intelligence is transforming education. ChatGPT helps students learn NLP effectively.

Token No.	Token
1	Artificial
2	Intelligence
3	is
4	transforming
5	education
6	.
7	ChatGPT
8	helps
9	students
10	learn
11	NLP
12	effectively
13	.

Total Tokens

✓ Word tokens: 11

✓ Punctuation tokens: 2

✓ Total tokens: 13

7. Types of Tokenization

7.1 Sentence Tokenization

Divides a document or paragraph into individual sentences.

Example: NLP is useful. It is used in AI.

Output: ['NLP is useful.', 'It is used in AI.']

7.2 Word Tokenization

Divides a sentence into words and punctuation marks.

Example: NLP is interesting!

Output: ['NLP', 'is', 'interesting', '!']

7.3 Character Tokenization

Divides text into individual characters.

Example: NLP → N | L | P

7.4 Subword Tokenization

Divides a word into smaller meaningful units. For example, unhappiness may be represented as un + happy + ness. Subword tokenization is widely used in modern language models because it can handle rare and new words.

8. Word Tokenization Challenges

8.1 Contractions

Example: I can't attend.

Possible tokenization: ['I', "can't", 'attend', '.'] or ['I', 'ca', "n't", 'attend', '.']

8.2 Hyphenated Words

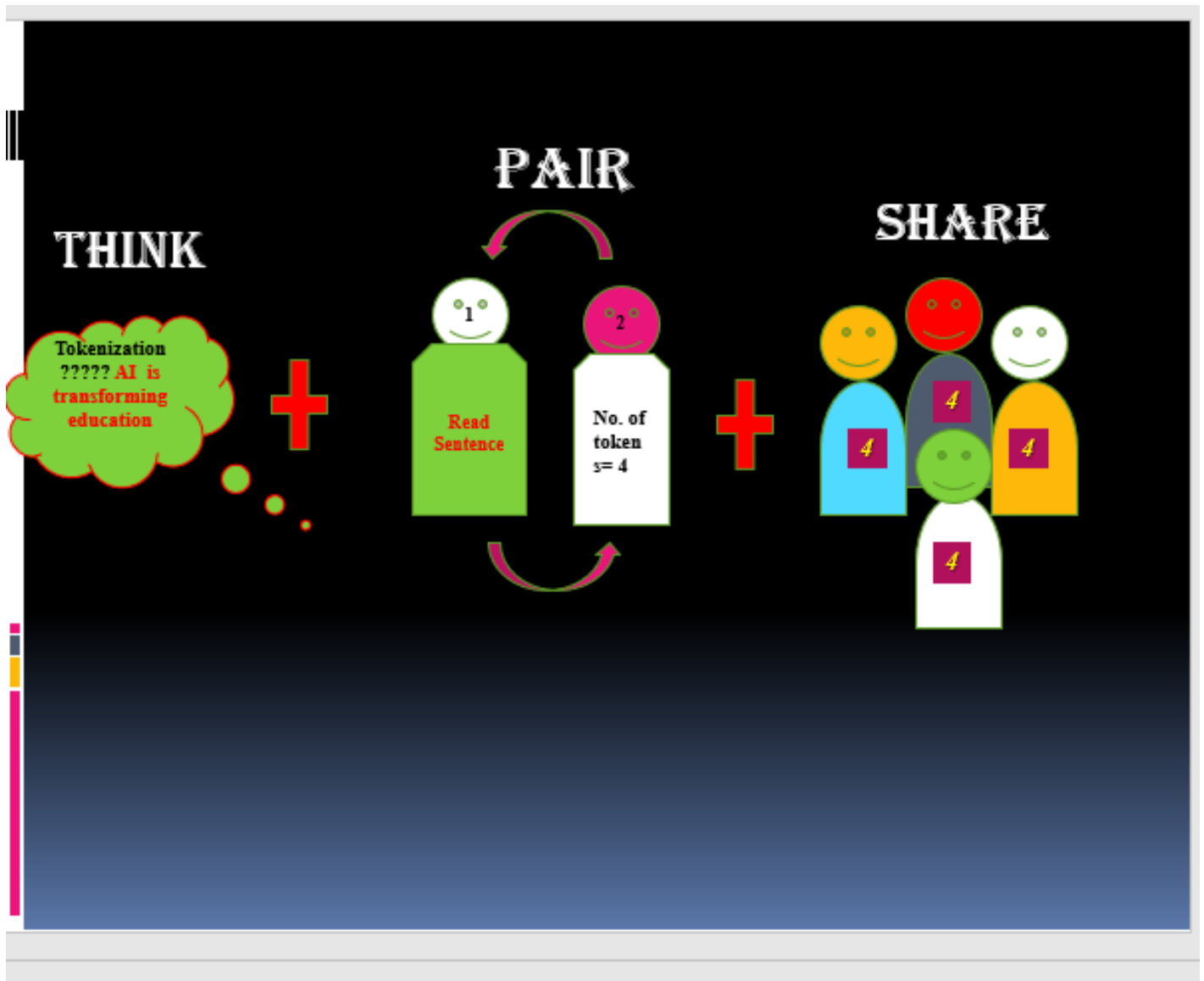
Example: state-of-the-art

Depending on the application, it may be treated as one token or split into individual words and hyphens.

8.3 Punctuation

Example: Hello, world!

Possible tokens: ['Hello', ',', 'world', '!']



9. Methods of Tokenization

Method	Description	Example	Advantages	Limitations
Whitespace Tokenization	Splits text wherever whitespace such as spaces, tabs, or new lines occurs.	Hello, world! → ['Hello,', 'world!']	Simple, fast, and easy to implement.	Punctuation may remain attached to words.
Rule-Based Tokenization	Uses predefined linguistic rules to identify spaces, punctuation, abbreviations, decimal numbers, URLs, email addresses, and	Dr. Smith scored 95.5 marks. → ['Dr.', 'Smith', 'scored', '95.5', 'marks', '.']	Handles special cases and punctuation more accurately.	Requires many carefully designed rules and may be language-dependent.

	contractions.			
Statistical Tokenization	Learns tokenization patterns from large collections of text called corpora and uses probability and context to determine token boundaries.	The system learns whether a period is part of an abbreviation or the end of a sentence.	Can learn complex language patterns and handle ambiguous cases.	Requires large training data and computational resources.

10. Methods of Sentence Splitting

10.1 Rule-Based Sentence Splitting

Rule-based systems use punctuation and language-specific rules. They must also recognize exceptions such as abbreviations and decimal numbers.

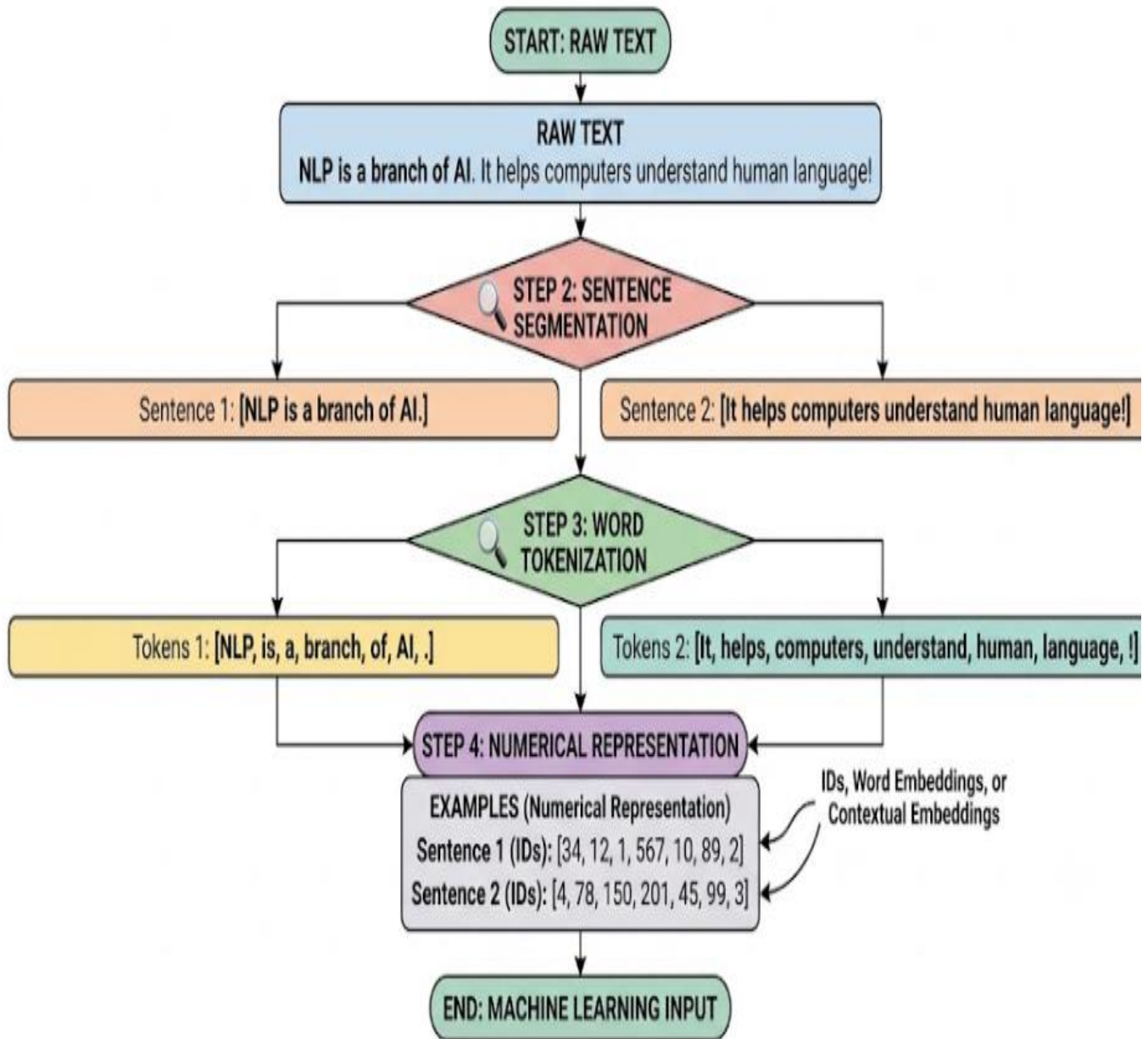
10.2 Machine Learning-Based Sentence Splitting

Machine learning models learn whether a punctuation mark represents a sentence boundary. Features may include the surrounding words, capitalization, abbreviation status, and contextual information.

Applications of Tokenization

- Search engines
- Sentiment analysis
- Machine translation
- Chatbots
- Speech recognition
- Text classification
- Information retrieval
- Large Language Models

COMPLETE TOKENIZATION PROCESS



Difference Between Text Segmentation and Sentence Splitting

Feature	Text Segmentation	Sentence Splitting
Meaning	Divides text into meaningful units	Divides text into individual sentences
Scope	Broad process	Specific process
Units	Paragraphs, sentences, words, subwords, characters	Sentences
Purpose	Organizes text structure	Identifies sentence boundaries
Example	Document → Paragraph → Sentence	Paragraph → Sentence 1, Sentence 2

MORPHOLOGICAL PARSING: STEMMING AND LEMMATIZATION

Morphological Parsing is the process of analyzing the internal structure of words to identify their root forms and affixes.

Original Word	Stemmed Word	Explanation
Playing	Play	The suffix -ing is removed to obtain the stem Play .
Studies	Studi	The suffix -es is removed, resulting in Studi , which is not a valid dictionary word .
Running	Run	The suffix -ing is removed, and the extra ' n ' is reduced to obtain the stem Run .

Aspect	Advantages	Disadvantages
Speed	Fast processing because it uses simple rules.	Does not analyze grammar or context.
Implementation	Easy to implement with rule-based algorithms.	May produce incorrect or incomplete stems.
Output	Reduces different word forms to a common stem, improving efficiency.	The output may not be a valid dictionary word (e.g., Studies → Studi).
Applications	Useful for search engines, information retrieval, and text mining.	Less suitable for applications requiring accurate language understanding, such as machine translation or question answering.
Accuracy	Efficient for matching related words in large datasets.	Ignores grammar and word meaning, reducing accuracy compared to lemmatization.

LEMMATIZATION

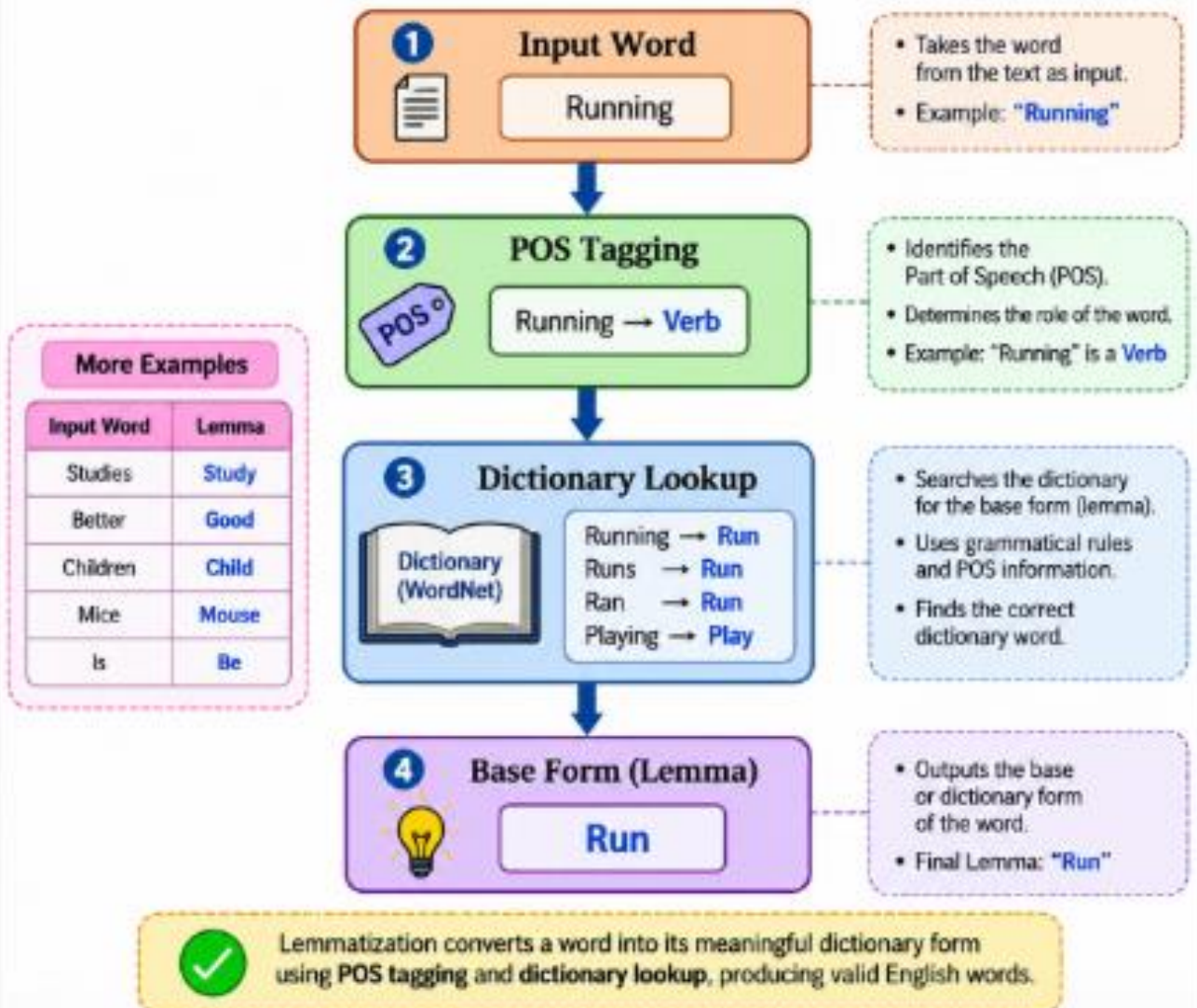
Lemmatization is the process of converting a word into its **base or dictionary form**, known as a **lemma**. Unlike stemming, lemmatization considers the **meaning, grammar, and Part-of-Speech (POS)** of a word before finding its root form.

In other words, lemmatization transforms different forms of a word into one meaningful base word that can be found in a dictionary.

Word	Lemma
Running	Run
Studies	Study
Better	Good
Children	Child
Mice	Mouse
Am	Be
Is	Be
Are	Be

Aspect	Advantages	Disadvantages
Output	Produces meaningful dictionary words (lemmas).	Processing is slower than stemming.
Accuracy	More accurate because it considers grammar and context.	Requires grammatical analysis and language rules.
Language Understanding	Preserves the actual meaning of words.	Needs more computational resources.
Resources	Improves the quality of NLP applications.	Requires a dictionary (lexicon) and Part-of-Speech (POS) tagging.
Applications	Suitable for chatbots, machine translation, sentiment analysis, and question answering.	More complex to implement than stemming.

Lemmatization Process



Feature	Stemming	Lemmatization
Definition	Removes prefixes or suffixes to obtain a stem.	Converts a word into its dictionary (base) form called a lemma.
Grammar	Does not consider grammar or context.	Considers grammar and Part-of-Speech (POS) information.
Dictionary	Does not require a dictionary.	Requires a dictionary (lexicon).
Output	May produce invalid or incomplete words.	Produces valid dictionary words.
Accuracy	Less accurate.	More accurate.
Speed	Faster.	Slower.
Example	Studies → Studi	Studies → Study
Best Used For	Search engines, indexing, information retrieval.	Chatbots, machine translation, text understanding, sentiment analysis.

Morphological Parsing: Stemming vs Lemmatization

Analagical Reasoning

Both are like finding the "root" of a word, but the way they do it is different.

STEMMING = Rough Cutting ✂



Imagine a gardener who cuts off the extra branches quickly to make the tree smaller.

Quick process | May remove extra parts
But sometimes, the result may not be perfect
(may not be a real word)



LEMMATIZATION = Careful Pruning ✂



Imagine a skilled gardener who understands each branch and trims it carefully to reach the original healthy root.

Slower process | Understands the tree well
Always gives the correct root (real word)

STEMMING

Stemming removes prefixes or suffixes from a word to get its stem. The stem **may not be a valid dictionary word**.



Word	After Stemming	Is it a real word?
Playing	Play	✓ Yes
Studies	Studi	✗ No
Running	Run	✓ Yes
Happiness	Happi	✗ No
Better	Better	✓ Yes

Advantages

- Fast
- Easy to implement
- Reduces word variations
- Useful in search engines and information retrieval

Disadvantages

- May produce invalid words (e.g., Studi, Happi)
- Ignores grammar and context
- Less accurate

Used in: Search Engines, Text Mining, Indexing, Autocomplete

LEMMATIZATION

Lemmatization converts a word to its base or dictionary form (lemma) using vocabulary and grammatical analysis. The lemma is **always a valid dictionary word**.

Word	After Lemmatization	Is it a real word?
Playing	Play	✓ Yes
Studies	Study	✓ Yes
Running	Run	✓ Yes
Happiness	Happy	✓ Yes
Better	Good	✓ Yes

Advantages

- Produces meaningful dictionary words
- More accurate
- Considers grammar and context
- Better for NLP applications

Disadvantages

- Slower than stemming
- Requires dictionary and POS tagging
- More complex to implement

Used in: Chatbots, Machine Translation, Spell Checkers, Text Understanding, Question Answering



KEY DIFFERENCE

Feature	Stemming	Lemmatization
Speed	Fast	Slower
Grammar	Not considered	Considered
Dictionary	Not required	Required
Output	May be invalid	Always valid
Accuracy	Lower	Higher

HOW LEMMATIZATION WORKS?



REAL LIFE APPLICATIONS

- Search Engines - better search results
- Chatbots - understand user queries
- Spell Checkers - correct suggestions
- Text Classification - better accuracy
- Machine Translation - accurate meaning

Spelling Error Detection and Correction

Spelling Error Detection is the process of identifying incorrectly spelled words.

Spelling Error Correction is the process of replacing the incorrect word with the most probable correct word.

Spelling Error Detection and Correction is a Natural Language Processing (NLP) task that identifies incorrectly spelled words in a text and suggests or automatically replaces them with the correct words.

It is widely used in applications such as **Grammarly, Microsoft Word Spell Checker, Google Docs, smartphone keyboards, search engines, and chatbots.**

Why is Spelling Correction Important?

People often make typing mistakes while entering text. These errors can affect communication and reduce the accuracy of NLP systems.

Incorrect Sentence	Correct Sentence
---------------------------	-------------------------

I hav a pen.	I have a pen.
She is lerning NLP.	She is learning NLP.
I recieved your email.	I received your email.

Types of Spelling Errors

1. Non-Word Error

A word that **does not exist** in the dictionary.

Incorrect	Correct
computar	computer
languge	language
recieve	receive

2. Real-Word Error

A word exists in the dictionary but is **used incorrectly in the sentence**.

Examples

Incorrect Sentence	Correct Sentence
I want too go.	I want to go.
Their coming today.	They're coming today.
He red the book.	He read the book.

These errors require understanding the sentence context, making them harder to detect.

Spelling Error Correction Process

The correction process generally consists of four steps:

Step 1: Detect the Error

Compare each word with a dictionary.

Example

Recive

Not found in the dictionary → Error detected.

Step 2: Generate Candidate Words

Possible corrections are generated.

Receive

Recipe

Recite

Receiver

Step 3: Rank the Candidates

Candidates are ranked using:

- Minimum Edit Distance
- Word frequency
- Language models
- Context

Step 4: Select the Best Correction

The system chooses the most probable word.

Recive



Receive

Common Error Correction Techniques

1. Dictionary-Based Method

Each word is checked against a dictionary.

Example

languge
↓
language

Advantages

- Simple
- Fast

Disadvantages

- Cannot detect real-word errors.

2. Minimum Edit Distance (MED)

The correct word is the one requiring the **fewest editing operations**.

Operations include:

- Insertion
- Deletion
- Substitution

Example

helo
↓
hello

Only one insertion is required.

3. Phonetic Matching

Corrects words based on similar pronunciation.

Examples

nite

↓

night

fone

↓

phone

Algorithms such as Soundex and Metaphone are commonly used.

4. Statistical Language Models

The system predicts the most likely word based on surrounding words.

Example

I want too eat.

Since "want to eat" is much more common than "want too eat," the model suggests:

I want to eat.

5. Deep Learning Models

Modern NLP systems use neural networks and Transformer models (such as BERT) to understand sentence context and suggest accurate corrections.

Example

He red the book.

The model recognizes that "read" fits the context better than "red."

Example

Input:

The studnts are lerning naturl language procesing.

Detection:

studnts
lerning
naturl
languge
procesing

Correction:

students
learning
natural
language
processing

Final sentence:

The students are learning natural language processing.

Applications

1. Grammarly

- Detects spelling mistakes
- Suggests corrections
- Improves writing quality

2. Microsoft Word Spell Checker

- Underlines misspelled words
- Provides correction suggestions
- Can automatically correct common errors

3. Smartphone Keyboards

- Autocorrect while typing
- Predictive text suggestions

4. Search Engines

- Correct misspelled search queries

Example

Googel



Google

5. Optical Character Recognition (OCR)

- Corrects spelling errors produced after scanning printed documents.

6. Chatbots and Voice Assistants

- Improve understanding of user input by correcting spelling mistakes before processing.
-

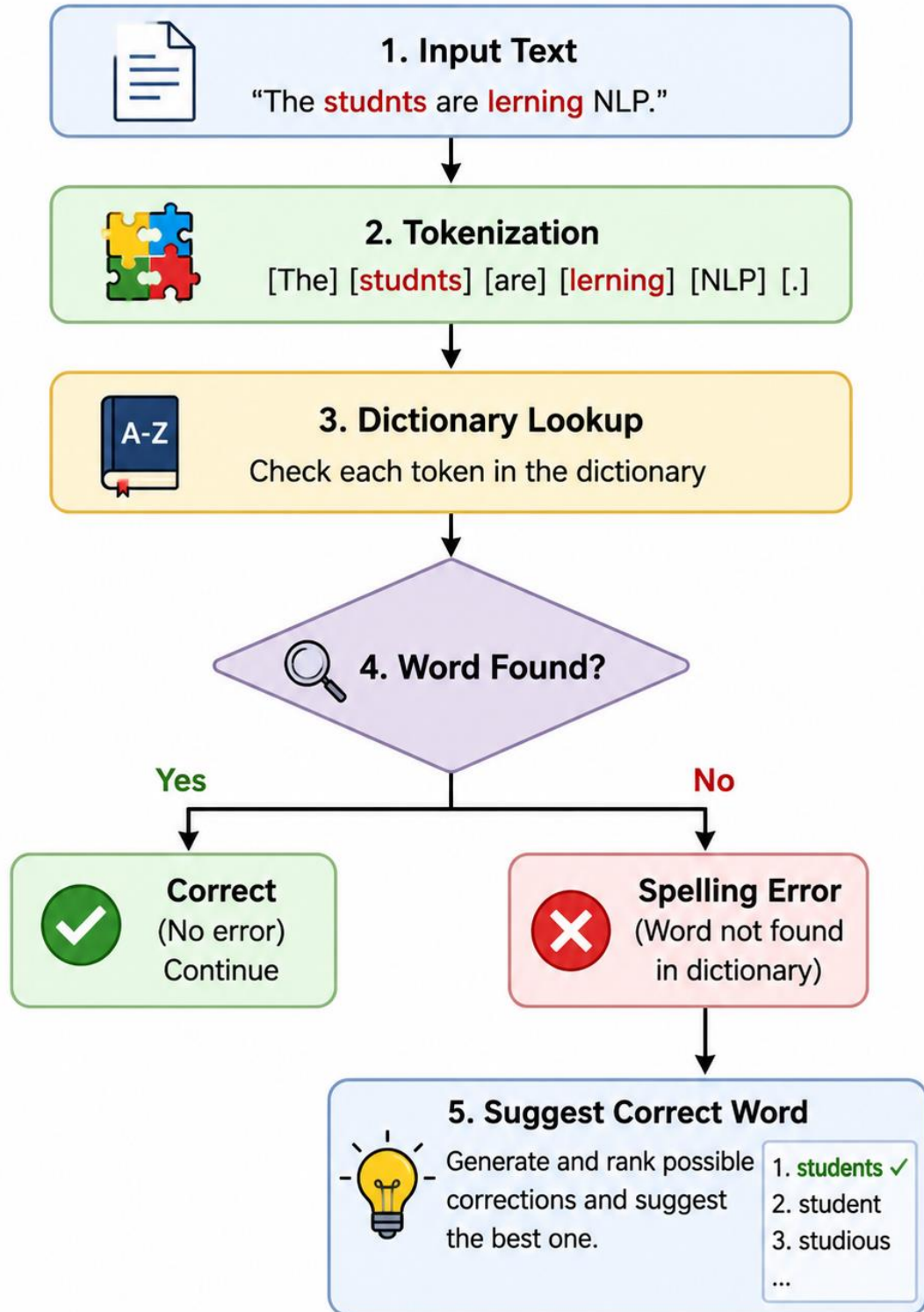
Advantages

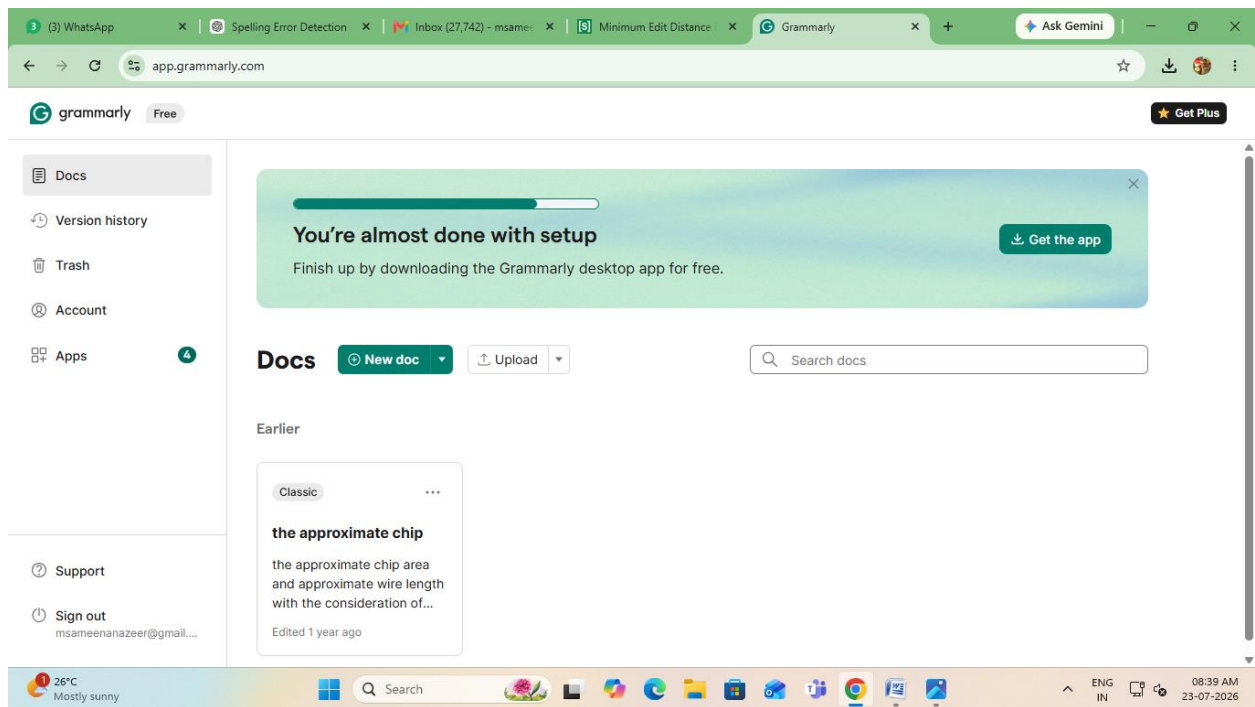
- Improves text quality
 - Reduces typing mistakes
 - Enhances search accuracy
 - Improves chatbot and voice assistant performance
 - Saves users time
-

Limitations

- Difficult to detect real-word errors
- May suggest incorrect replacements
- Performance depends on dictionary quality and context
- Domain-specific terms may not be recognized

Steps in Spelling Error Detection





What is Minimum Edit Distance (MED)?

Definition

Minimum Edit Distance (MED) is the **minimum number of editing operations** required to convert one word (or string) into another.

The three basic editing operations are:

- **Insertion** – Add a character
- **Deletion** – Remove a character
- **Substitution** – Replace one character with another

Allowed Operations

Operation	Example	Cost
Insertion	CAT → CHAT (Insert H)	1
Deletion	BOOKS → BOOK (Delete S)	1
Substitution	PEN → PIN (Replace E with I)	1

How MED is Calculated

1. Compare the two words character by character.

2. If characters are the same, no edit is needed.
3. If characters differ, choose the operation (Insertion, Deletion, or Substitution) with the lowest cost.
4. Continue until the first word becomes the second word.
5. The total number of edits is the **Minimum Edit Distance**.

• Real-Time Applications

- **1. Smartphone Autocorrect**

- User types:
 - **helo**
 - Phone suggests:
 - **hello**
 - because the edit distance is **1** (insert l).
-

- **2. DNA Sequence Matching**

- Scientists compare DNA sequences by calculating the minimum insertions, deletions, and substitutions needed to transform one sequence into another.
- **Protein Structure Analysis:** Compares amino acid sequences to identify structural similarities and evolutionary relationships.

- **3. Spell Checkers**

- Applications such as **Microsoft Word** and **Grammarly** compare misspelled words with dictionary words and choose the one with the smallest edit distance.
-

- **4. Search Engines**

- If a user searches for:
- **googel**
- the search engine suggests:
- **google**
- because it has the minimum edit distance.

MINIMUM EDIT DISTANCE AND APPLICATIONS

— Analogical Reasoning Approach —



DEFINITION

Minimum Edit Distance (MED) between two strings is the minimum number of operations required to transform one string into another.

Operations allowed:

1. Insertion
2. Deletion
3. Substitution

ANALOGY



Think of words as sentences written on paper.

To change one word into another, we can:

Insert a letter, delete a letter, or replace a letter.

We want the least number of such changes.



ALLOWED OPERATIONS

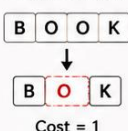
1. INSERTION

Add a letter



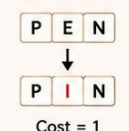
2. DELETION

Remove a letter



3. SUBSTITUTION

Replace a letter



SIMPLE EXAMPLE

Transform "CAT" to "CUT"

C A T → C U T

Replace A → U

MED = 1

ANOTHER EXAMPLE

Transform "BOOK" to "BACK"

B O O K → B A C K

BOOK → BAOK (Replace O → A)

BAOK → BACK (Replace O → C)

MED = 2

HOW MED IS CALCULATED? (Dynamic Programming Table)

Example:
Transform
"kitten"
to
"sitting"

	∅	i	t	t	i	n	g
∅	0	1	3	4	5	6	7
k	1	1	3	4	5	6	7
i	2	2	1	3	4	5	6
t	3	3	1	2	3	4	5
t	4	4	2	1	2	3	4
e	5	5	3	2	2	3	4
n	6	6	5	3	3	2	3

Minimum
Edit
Distance
= 3

One optimal path: kitten → sitten (substitute k→s)
sitten → sittin (substitute e→i)
sittin → sitting (insert g)

APPLICATIONS



1. AUTOCORRECT IN SMARTPHONES

When we type "helo", the phone suggests "hello" because the edit distance between them is 1.



2. DNA SEQUENCE MATCHING

Used in bioinformatics to find similarity between DNA/RNA sequences by minimum insertions, deletions and substitutions.



3. SPELL CHECKER IN SEARCH ENGINES

If we search "googel", engine shows "google" as the closest match.



4. CHATBOTS AND NLP SYSTEMS

Helps in understanding user input with typos and providing correct responses.



5. PLAGIARISM DETECTION

Measures similarity between documents/sentences allowing small variations.



Key Takeaway: Minimum Edit Distance helps computers measure how similar two strings are and is a powerful tool used in many real-world NLP and bioinformatics applications.



Statistical Language Models

A Statistical Language Model (SLM) is a mathematical model that assigns probabilities to sequences of words. It predicts the next word in a sentence based on the occurrence of previous words in a large text corpus.

Why Do We Need Language Models?

Language models help computers:

- Predict the next word in a sentence.
- Determine whether a sentence is grammatically correct.
- Improve text input through auto-complete.
- Translate text between languages.
- Generate human-like text.

Example:

Input:

I am going to the _____

Possible predictions:

- market ✓
- school ✓
- moon ✕(less likely depending on context)

What is an N-gram?

An **N-gram** is a sequence of **N consecutive words**.

Model	Number of Words
Unigram	1 word
Bigram	2 words
Trigram	3 words

1. Unigram Model

Definition

A **Unigram model** assumes that each word occurs independently of every other word. The probability of a sentence is calculated by multiplying the probabilities of individual words.

Formula

$$P(w_1, w_2, \dots, w_n) = P(w_1) \times P(w_2) \times \dots \times P(w_n)$$

Example Corpus

I love NLP
I love AI
I study NLP

Word Frequency

Word	Count
I	3
love	2
NLP	2
AI	1
study	1

Total words = 9

Word	Probability
I	$3/9 = 0.333$
love	$2/9 = 0.222$
NLP	$2/9 = 0.222$
AI	$1/9 = 0.111$
study	$1/9 = 0.111$

Example

Probability of:

I love NLP

$$P = P(I) \times P(\text{love}) \times P(\text{NLP}) = 0.0164$$

Advantages

- Simple to implement.
- Requires less memory.
- Fast computation.

Disadvantages

- Ignores context.
- Produces poor predictions.

2. Bigram Model

A **Bigram model** predicts a word using the immediately preceding word.

Formula

$$P(w_i|w_{i-1})$$

Example Corpus

I love NLP

I love AI

I study NLP

Bigram	Count
I love	2
love NLP	1
love AI	1
I study	1
study NLP	1

Probability Calculation

Probability of love after I

$$\begin{aligned}P(\text{love}|\text{I}) &= \frac{\text{Count}(\text{I love})}{\text{Count}(\text{I})} \\&= \frac{2}{3} = 0.667\end{aligned}$$

Probability of NLP after love

$$P(\text{NLP}|\text{love}) = \frac{1}{2} = 0.5$$

Example Prediction

Input

I love _____

Predictions:

- NLP
- AI

Both have equal probability.

Advantages

- Considers previous word.
- Better accuracy than Unigram.

Disadvantages

- Limited context.
- Suffers from data sparsity.

3. Trigram Model

A **Trigram model** predicts the next word using the previous **two words**.

Formula

$$P(w_i | w_{i-2}, w_{i-1})$$

Example Corpus

I love NLP

I love AI

I study NLP

Trigrams

Trigram

I love NLP

I love AI

I study NLP

Prediction

Input

I love _____

Possible predictions:

- NLP
- AI

If another sentence is added:
I love Machine Learning

The trigram model uses the context I love to predict the most probable continuation.

Advantages

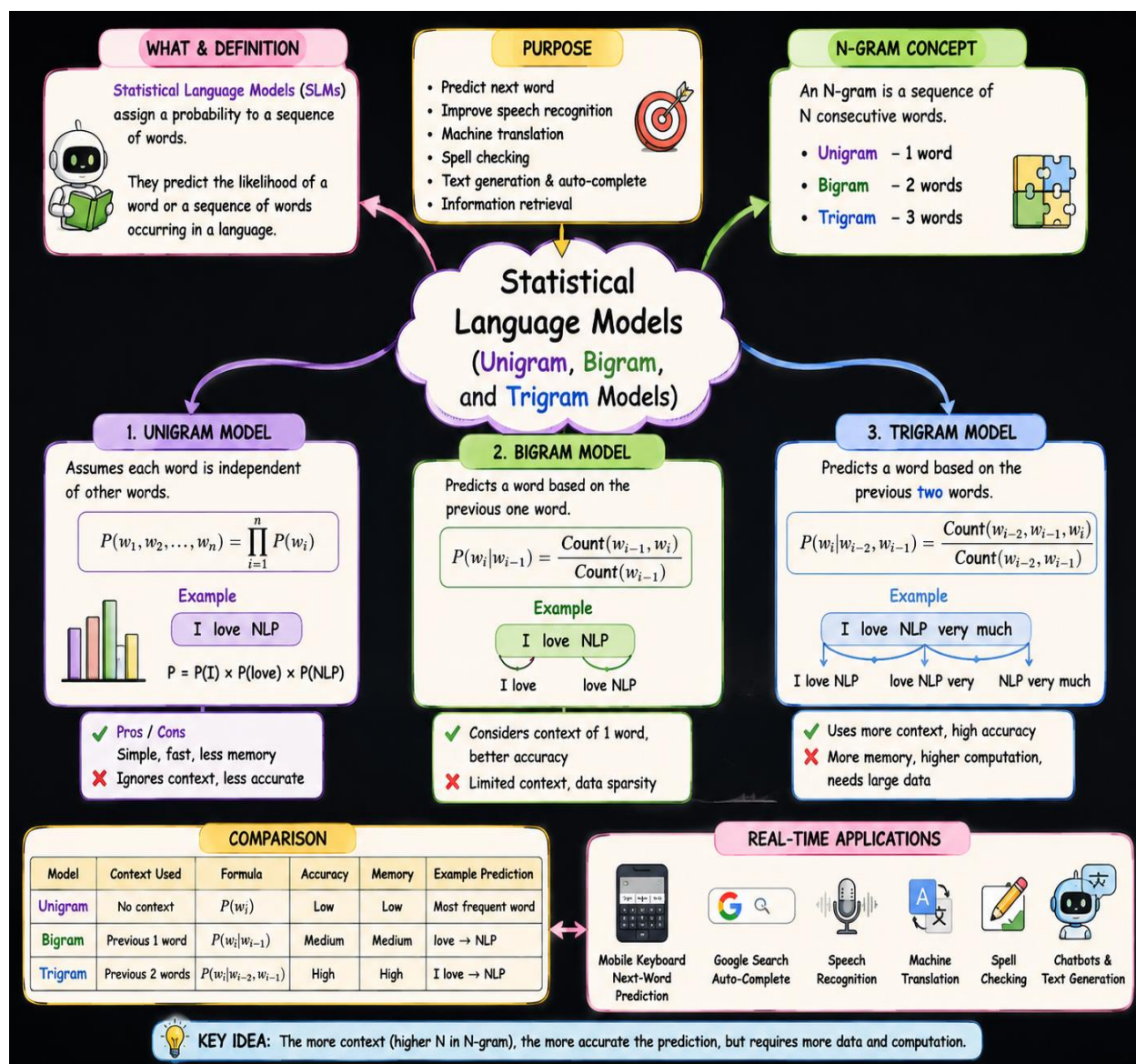
- Uses more context.
- Produces better predictions.
- More accurate than Unigram and Bigram.

Disadvantages

- Requires a larger dataset.
- Higher memory and computational cost.

Feature	Unigram	Bigram	Trigram
Context Used	None	Previous 1 word	Previous 2 words
Prediction Accuracy	Low	Medium	High
Memory Requirement	Low	Medium	High
Computational Complexity	Low	Medium	High
Example	NLP	love → NLP	I love → NLP

Application	Language Model Used
Mobile keyboard next-word prediction	Bigram, Trigram
Google Search auto-complete	Bigram, Trigram
Speech recognition	N-gram models
Machine translation	N-gram models
Spell checking	Unigram, Bigram
Chatbots	Trigram and higher-order models
Email text prediction	Bigram, Trigram
Voice assistants (Siri, Alexa, Google Assistant)	N-gram models



Bigram Count – Step-by-Step Explanation

Corpus:

Sentence 1: I love NLP

Sentence 2: I love AI

Step 1: List all Bigrams

(I, love)

(love, NLP)

(I, love)

(love, AI)

Step 2: Count Each Bigram

Bigram	Count
(I, love)	2
(love, NLP)	1
(love, AI)	1

Step 3: Count Individual Words

Word	Count
I	2
love	2
NLP	1
AI	1

Formula

$$P(w_n | w_{(n-1)}) = \text{Count}(w_{(n-1)}, w_n) / \text{Count}(w_{(n-1)})$$

Calculations

1. $P(\text{love} | I) = \text{Count}(I, \text{love}) / \text{Count}(I) = 2/2 = 1.0$

Meaning: Every occurrence of "I" is followed by "love".

2. $P(\text{NLP} | \text{love}) = \text{Count}(\text{love}, \text{NLP}) / \text{Count}(\text{love}) = 1/2 = 0.5$

Meaning: After "love", NLP appears in one of two cases.

3. $P(\text{AI} | \text{love}) = \text{Count}(\text{love}, \text{AI}) / \text{Count}(\text{love}) = 1/2 = 0.5$

Meaning: After "love", AI appears in one of two cases.

Final Answers

Probability	Value
$P(\text{love} I)$	1.0
$P(\text{NLP} \text{love})$	0.5
$P(\text{AI} \text{love})$	0.5

Trigram Model in Natural Language Processing (NLP)

Definition

A **Trigram Model** is a statistical language model that predicts the **next word based on the previous two words**.

It is called a **3-gram model** because it considers **three consecutive words** at a time. The lecture material describes trigram models as extending the n-gram approach by using more context than bigrams.

Formula

The probability of the current word depends on the previous **two** words.

$$P(w_n | w_{n-2}, w_{n-1}) = \frac{C(w_{n-2}, w_{n-1}, w_n)}{C(w_{n-2}, w_{n-1})}$$

Where:

- $C(w_{n-2}, w_{n-1}, w_n)$ = Count of the trigram
- $C(w_{n-2}, w_{n-1})$ = Count of the previous two-word sequence

Example

Corpus

I love machine learning
I love deep learning

Step 1: Generate Trigrams

Sentence 1

I love machine learning

Trigrams

(I, love, machine)

(love, machine, learning)

Sentence 2

I love deep learning

Trigrams

(I, love, deep)

(love, deep, learning)

Step 2: Count Trigrams

Trigram	Count
(I, love, machine)	1
(love, machine, learning)	1
(I, love, deep)	1
(love, deep, learning)	1

Step 3: Count Bigrams

Since trigram probability uses the count of the previous **two words**, count the bigrams.

Bigram	Count
(I, love)	2
(love, machine)	1
(love, deep)	1

Example 1

Find

$P(\text{machine} | \text{I, love})$

Step 1

Count the trigram

(I, love, machine)

Count = **1**

Step 2

Count the bigram

(I, love)

Count = **2**

Step 3

Apply the formula

$$P(\textit{machine} | I, \textit{love}) = \frac{1}{2} = 0.5$$

Answer

$$P(\textit{machine} | I, \textit{love}) = 0.5$$

Interpretation: After the words "I love", the word "machine" appears in **1 out of 2 cases**, so the probability is **0.5 (50%)**.

Bigram Model – Different Example

Definition: A Bigram Model predicts the next word using the previous one word.

Formula: $P(w_n | w_{(n-1)}) = \text{Count}(w_{(n-1)}, w_n) / \text{Count}(w_{(n-1)})$

Corpus

Data Science is useful

Data Science is interesting

Machine Learning is useful

Step 1: Generate Bigrams

(Data, Science)

(Science, is)

(is, useful)

(Data, Science)

(Science, is)

(is, interesting)

(Machine, Learning)

(Learning, is)

(is, useful)

Bigram	Count
(Data, Science)	2
(Science, is)	2
(is, useful)	2
(is, interesting)	1
(Machine, Learning)	1
(Learning, is)	1

Word Counts

Word	Count
Data	2
Science	2
is	3
useful	2
interesting	1
Machine	1
Learning	1

Solved Examples

1. $P(\text{Science} | \text{Data}) = \text{Count}(\text{Data}, \text{Science}) / \text{Count}(\text{Data}) = 2/2 = 1.0$

2. $P(\text{is} | \text{Science}) = \text{Count}(\text{Science}, \text{is}) / \text{Count}(\text{Science}) = 2/2 = 1.0$

3. $P(\text{useful} | \text{is}) = \text{Count}(\text{is}, \text{useful}) / \text{Count}(\text{is}) = 2/3 = 0.67$

4. $P(\text{interesting} | \text{is}) = \text{Count}(\text{is}, \text{interesting}) / \text{Count}(\text{is}) = 1/3 = 0.33$

Probability	Calculation	Answer
-------------	-------------	--------

$P(\text{Science} \text{Data})$	2/2	1.0
$P(\text{is} \text{Science})$	2/2	1.0
$P(\text{useful} \text{is})$	2/3	0.67
$P(\text{interesting} \text{is})$	1/3	0.33

Interpretation

Every occurrence of "Data" is followed by "Science", so the probability is 1.0. After "is", the next word is "useful" two out of three times (0.67) and "interesting" one out of three times (0.33).

Trigram Model – Another Example

Definition: A Trigram Model predicts the next word using the previous two words. It uses three consecutive words (3-grams) to calculate probabilities.

Formula: $P(w_n | w_{n-2}, w_{n-1}) = \text{Count}(w_{n-2}, w_{n-1}, w_n) / \text{Count}(w_{n-2}, w_{n-1})$

Corpus

Students study NLP
Students study Python
Teachers teach NLP

Generated Trigrams

(Students, study, NLP)
(Students, study, Python)
(Teachers, teach, NLP)

Trigram	Count
(Students, study, NLP)	1
(Students, study, Python)	1
(Teachers, teach, NLP)	1

Bigram	Count
(Students, study)	2
(Teachers, teach)	1

Solved Problems

1. $P(\text{NLP} | \text{Students study}) = \text{Count}(\text{Students, study, NLP}) / \text{Count}(\text{Students, study}) = 1/2 = 0.5$
2. $P(\text{Python} | \text{Students study}) = \text{Count}(\text{Students, study, Python}) / \text{Count}(\text{Students, study}) = 1/2 = 0.5$
3. $P(\text{NLP} | \text{Teachers teach}) = \text{Count}(\text{Teachers, teach, NLP}) / \text{Count}(\text{Teachers, teach}) = 1/1 = 1.0$

Probability	Calculation	Answer
$P(\text{NLP} \text{Students, study})$	1/2	0.5
$P(\text{Python} \text{Students, study})$	1/2	0.5
$P(\text{NLP} \text{Teachers, teach})$	1/1	1.0

Interpretation

After "Students study", there are two possible next words: NLP and Python, so each has probability 0.5. After "Teachers teach", the only observed next word is NLP, so its probability is 1.0 (100%).